Design Q&A Support Agent

題目描述（Problem Statement）

設計一個 AI 驅動的客服問答代理（Q&A Support Agent），部署於訂房平台（如 Booking.com、Agoda）上，讓使用者能以自然語言詢問與訂房相關的問題，並獲得準確、即時的回答。

系統需要能夠處理多種類型的問題，例如：

- **平台通用問題**：「如何取消訂房？」、「退款政策是什麼？」
- **飯店特定問題**：「這間飯店可以帶寵物嗎？」、「check-in 時間是幾點？」
- **使用者訂單問題**：「我的訂單目前是什麼狀態？」、「我還能免費取消嗎？」

核心挑戰在於：**答案必須基於平台的真實資料（知識庫、訂單紀錄、飯店政策），而非 LLM 的預訓練知識**。系統需要透過 RAG（Retrieval-Augmented Generation）架構，在回答前先檢索相關資料，再基於檢索結果生成回答，確保答案的準確性與可追溯性。

buildmoat.org

功能性需求（Functional Requirements）

1. 使用者應該能夠以自然語言提問，並取得基於平台知識庫的準確回答。
2. 系統應該能夠從多種資料來源（FAQ、飯店政策、使用者訂單）中檢索相關資訊，作為生成回答的依據。
3. 系統應支援多輪對話（multi-turn conversation），能理解上下文脈絡。
4. 系統應該能夠在無法回答時，明確告知使用者並提供轉接真人客服的選項。

非功能性需求（Non-Functional Requirements）

1. **低延遲（Low latency）**：使用者從提問到收到回答，端到端延遲應控制在 2-3 秒以內。

2. **高準確性（High accuracy）**：回答必須基於檢索到的文件（grounded），不得產生幻覺（hallucination），例如捏造不存在的政策或虛構的訂單狀態。
3. **高可用性（High availability）**：客服系統是使用者遇到問題時的第一接觸點，需保持高可用。
4. **知識即時性（Knowledge freshness）**：當飯店政策更新或 FAQ 變動時，系統應該能夠在合理時間內反映最新資訊。

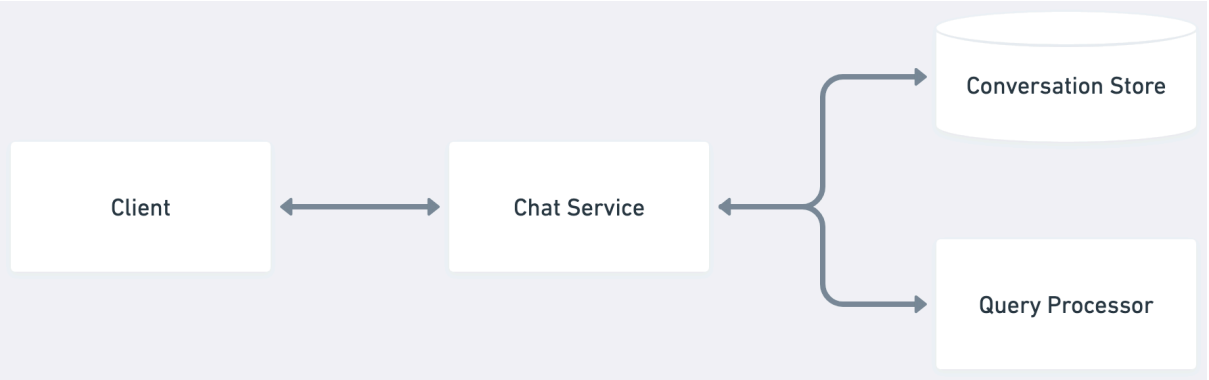
API Design

```
POST /chat

Body: {
  session_id,   // optional，首次對話可省略，由系統產生
  user_id,
  message
}

Response: {
  session_id,
  response,
  sources: [     // 回答所引用的資料來源，供使用者驗證
    { title, url, snippet }
  ]
}
```

buildmoat.org

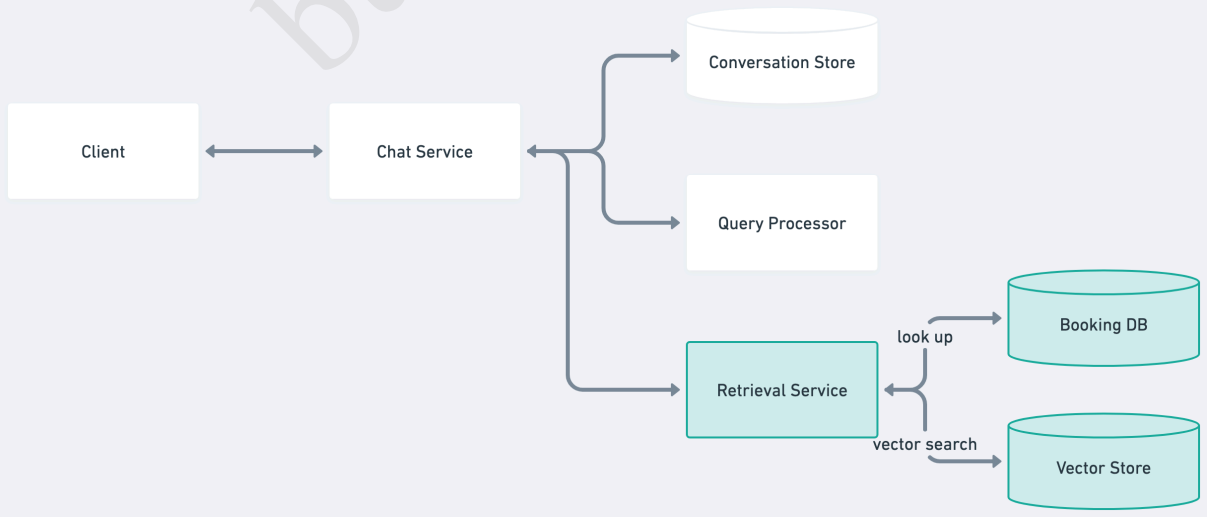
High-Level Design

1. 使用者提問，系統理解意圖



- 使用者透過 `POST /chat` 送出訊息至 **Chat Service**。
- Chat Service 根據 `session_id` 從 **Conversation Store** 中取得對話歷史（若為首次對話則建立新 session）。
- Chat Service 將使用者訊息與對話歷史傳給 **Query Processor**。
- Query Processor 執行兩項工作：
  - **Intent detection**：判斷問題類型——是通用 FAQ、飯店特定政策，還是使用者訂單查詢。這決定了後續要查詢哪些資料來源。
  - **Query rewriting**：結合對話歷史，將使用者的問題改寫為一個 self-contained 的查詢。例如使用者先問「台北有哪些飯店？」，再問「最便宜的那間可以帶狗嗎？」，改寫後的查詢應為「台北最便宜的飯店是否允許攜帶寵物？」。

2. 系統從多個資料來源檢索相關資訊



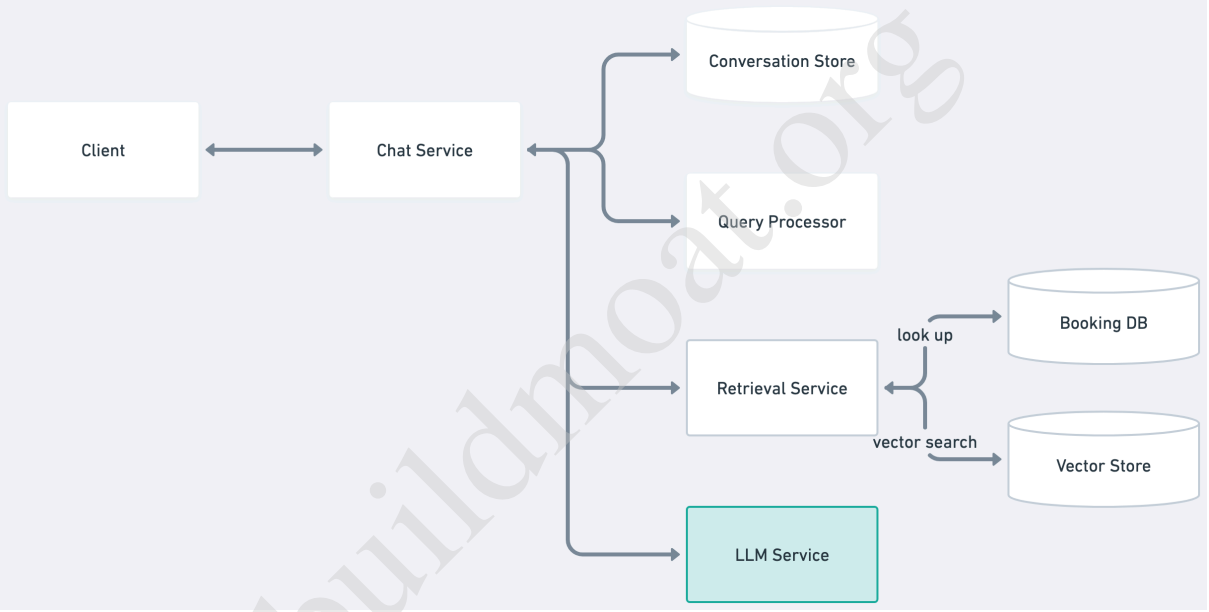
- **Retrieval Service** 根據 Query Processor 判斷的 intent，決定查詢哪些資料來源：

buildingai.org

- **非結構化知識**（FAQ、飯店政策、幫助文件）：透過 **Vector Search** 從 Vector Store 中檢索語意相近的文件片段（chunks）。
- **結構化訂單資料**（訂單狀態、付款紀錄、入住日期）：透過 **Booking DB Lookup** 以 `user_id` 或 `booking_id` 查詢訂單資料庫。
- 部分問題可能同時需要兩者。例如「我還能免費取消嗎？」需要查詢使用者的訂單（取得訂房日期與飯店 ID）以及該飯店的取消政策（取得免費取消截止期限）。

- 檢索結果經過 **ranking** 後，選取 top-K 最相關的 chunks 作為 context。

3. 系統基於檢索結果生成回答



buildmoat.org

- **LLM Service** 接收以下資訊，組成 prompt 並生成回答：
  - System prompt（定義角色、行為規範、回答格式）。
  - 檢索到的 context（relevant chunks + 訂單資料）。
  - 對話歷史（前幾輪的問答）。
  - 使用者的當前問題。
- LLM 基於 context 生成回答。若 context 中不包含足以回答問題的資訊，LLM 應回覆「無法回答」，並建議使用者聯繫真人客服。
- 回答連同引用的 sources 一起回傳給使用者。

深入探討（Deep Dives）

1. 我們要如何對知識庫文件進行分塊（chunking）與索引（indexing），以確保有效檢索？

High-level Design 只有包含系統的 online path。除此之外，系統還需要一個離線的 **Ingestion Pipeline**，負責將知識庫文件處理後寫入 Vector Store：

Chunking 策略直接影響檢索品質。chunk 太大，可能包含過多不相關資訊，稀釋語意；chunk 太小，可能失去上下文，導致檢索結果無法理解。

Chunking 策略的選擇：

策略	做法	適合場景
Fixed-size chunking	固定字數（如 500 字）切分，相鄰 chunk 有重疊（如 100 字 overlap）	通用、簡單，適合結構不明確的文件
Structure-based chunking	依據文件結構切分（如 FAQ 的每個 Q&A pair 為一個 chunk，政策文件的每個段落為一個 chunk）	結構化文件，如 FAQ、條款
Semantic chunking	依據語意邊界切分（句子間語意相似度下降時切割）	長篇、主題多樣的文件

訂房平台的建議做法：

- **FAQ 文件**：每個 Q&A pair 自然形成一個 chunk。這類文件結構清晰，使用 structure-based chunking 最為直覺。
- **飯店政策文件**：按段落或政策條目切分，並在每個 chunk 的 metadata 中標註 `hotel_id`、`policy_type`（cancellation / pet / check-in 等），以便檢索時可按 metadata 過濾。
- **幫助文件 / 長篇指南**：使用 fixed-size chunking with overlap，確保不會因為切割邊界而遺失上下文。

Embedding 與索引：

- 每個 chunk 經過 embedding model 轉換為向量後，連同 metadata 一起寫入 Vector Store（如 Pinecone、Weaviate、pgvector）。

buildnoat.org

- 檢索時，將使用者的 query 同樣轉為向量，透過 approximate nearest neighbor（ANN）搜尋找出最相近的 top-K chunks。

2. 使用者的問題同時需要結構化與非結構化資料時，如何處理？

許多實際問題需要同時查詢多個資料來源。例如：

```
使用者：「我還能免費取消嗎？」

需要的資訊：
├── 結構化資料（Booking DB）
|    └── 使用者的訂房紀錄：入住日期、飯店 ID、訂房方案
|
└── 非結構化資料（Vector Store）
     └── 該飯店的取消政策：免費取消截止日為入住前 X 天
```

解法：Query Router + 平行檢索

- Query Processor 分析使用者問題後，將 intent 分類為以下之一：
  - `KNOWLEDGE_ONLY`：純知識查詢（如「退款通常需要幾天？」）→ 僅查詢 Vector Store。
  - `BOOKING_ONLY`：純訂單查詢（如「我的訂單編號是什麼？」）→ 僅查詢 Booking DB。
  - `HYBRID`：混合查詢（如「我還能免費取消嗎？」）→ 同時查詢兩者。
- 對於 `HYBRID` 類型，Retrieval Service 可**平行**發出兩個查詢（Vector Search + DB Lookup），將兩者結果合併後一起送入 LLM 作為 context。
- Intent 分類本身可透過 LLM（以 few-shot prompting 實現）或輕量級分類模型完成。

3. 我們要如何防止 LLM 產生幻覺（hallucination），確保回答的準確性？

在客服場景中，幻覺的後果特別嚴重——若 LLM 捏造一個不存在的取消政策，使用者據此操作後發現不符，將導致信任危機與客訴。

多層防護機制：

第一層：Prompt engineering — 約束 LLM 的行為

buildmoat.org

- 在 system prompt 中明確指示：「你只能根據提供的 context 回答問題。如果 context 中沒有足夠資訊，請回覆『我無法確定，建議您聯繫真人客服』。不得猜測或捏造任何政策、日期或金額。」
- 要求 LLM 在回答中引用具體的 source（例如「根據該飯店的取消政策.....」），使回答可追溯。

第二層：Retrieval quality — 確保 LLM 收到正確的 context

- 若檢索結果不相關，LLM 即使遵守指示也無法給出好答案。因此檢索品質是防止幻覺的根本。
- 使用 **hybrid search**（向量搜尋 + 關鍵字搜尋）提升 recall：向量搜尋擅長捕捉語意相似性，關鍵字搜尋擅長精確匹配（如飯店名稱、政策編號）。
- 對檢索結果進行 **reranking**（使用 cross-encoder 模型），將最相關的 chunks 排在前面，減少 LLM 被不相關 context 干擾的機率。

第三層：Output validation — 檢查 LLM 的回答

- 對 LLM 的回答進行 **grounding check**：驗證回答中的關鍵事實（日期、金額、政策內容）是否出現在檢索到的 context 中。
- 可透過另一個 LLM call 或規則引擎實現：「以下回答是否完全基於提供的 context？有無包含 context 中未提及的事實？」
- 若 grounding check 失敗，回退至安全回答（「建議您聯繫真人客服」）。

第四層：Fallback — 明確的回退機制

- 當檢索結果的相關性分數（similarity score）低於閾值時，系統應主動拒絕回答，而非將低品質的 context 交給 LLM。
- 提供明確的轉接選項：「我無法確認這個問題的答案，是否需要我為您轉接真人客服？」

4. 我們要如何處理多輪對話（multi-turn conversation）？

使用者的對話通常不是單一問題，而是連續的追問：

```
使用者：「台北有哪些飯店推薦？」
Agent  ：「推薦 A 飯店、B 飯店.....」
使用者：「第一間的取消政策是什麼？」          ← "第一間" 需要從上文推斷
使用者：「那可以帶寵物嗎？」                  ← "那" 指的仍是 A 飯店
```

buildnoat.org

挑戰： 使用者的後續問題通常包含代詞（「那間」、「它」）或省略主語，直接將這樣的問題送入檢索系統，會因為缺乏上下文而檢索不到相關文件。

解法：Conversation-aware query rewriting

- **Conversation Store** 儲存每個 session 的完整對話歷史（使用者訊息 + agent 回覆）。
- 每次使用者提問時，Query Processor 將對話歷史與當前問題一起送入 LLM，要求改寫為 self-contained 的查詢：
  - 原始問題：「那可以帶寵物嗎？」
  - 改寫後：「台北 A 飯店是否允許攜帶寵物入住？」
- 改寫後的查詢再送入 Retrieval Service，確保檢索品質不受代詞或省略的影響。

對話歷史的管理：

- 對話歷史過長會增加 LLM 的 token 消耗與延遲。可設定 sliding window（例如只保留最近 10 輪對話），或使用摘要機制將早期對話壓縮為簡短摘要。
- Conversation Store 可使用 Redis（TTL 自動過期）或關聯式資料庫（需持久保存對話紀錄時）。

5. 我們要如何確保知識庫的即時性（knowledge freshness）？

飯店政策可能隨時更新（例如旺季調整取消政策、新增設施），FAQ 也會持續新增。若知識庫未及時更新，使用者會收到過時的回答。

Ingestion Pipeline 的設計：

- 知識來源（FAQ 系統、飯店管理後台、幫助中心 CMS）在內容變更時，透過 webhook 或 change event 通知 **Ingestion Pipeline**。
- Ingestion Pipeline 對變更的文件重新執行 chunking → embedding → 寫入 Vector Store 的流程。
- 為避免大量同時更新造成 pipeline 過載，使用 queue 緩衝更新事件，依序處理。

版本管理與一致性：

- 每個 chunk 在 Vector Store 中標記 `version` 與 `last_updated` metadata。
- 更新時採用 **write-then-delete** 策略：先寫入新版本的 chunks，確認成功後再刪除舊版本，避免更新過程中出現知識空窗。

buildmoat.org

- 對於時效性特別敏感的資訊（如促銷活動截止日期），可在 chunk 的 metadata 中標記 `expires_at`，檢索時自動過濾已過期的 chunks。

監控與告警：

- 追蹤知識庫中各文件的 last_updated 時間。若某份文件超過預設的更新週期仍未更新（例如飯店政策超過 90 天未更新），發出告警提醒內容團隊確認。
- 監控 retrieval 階段中「無相關結果」的比例。若此比例突然上升，可能代表有新類型的問題尚未被知識庫覆蓋，需要新增對應的文件。

6. Client 與內部服務之間的通訊協定如何選擇？

系統的通訊分為兩層：外部（Client ↔ Chat Service）與內部（Chat Service ↔ 其他服務），兩者的需求截然不同。

Client ↔ Chat Service：WebSocket vs SSE

	WebSocket	SSE（Server-Sent Events）
通訊方向	全雙工（bidirectional）	單向（server → client）
LLM streaming response	✅	✅
Server 主動推播（typing indicator、轉接通知）	✅ 原生支援	❌ 需額外機制
實作複雜度	較高（需管理連線生命週期、heartbeat、斷線重連）	較低（建立在標準 HTTP 上）
Load Balancer 相容性	需支援 WebSocket（sticky session 或 L4 LB）	標準 HTTP 即可

- 若產品只需「使用者問 → agent 逐字回覆」的簡單互動，**SSE** 足夠且更簡單——這也是 ChatGPT、Claude 等產品採用的方式。
- 若產品需要豐富的即時互動（typing indicator、轉接真人客服通知、agent 主動推播），**WebSocket** 更合適。
- 訂房平台的客服 agent 通常需要轉接真人客服等推播功能，因此偏向 **WebSocket**。

Chat Service ↔ 內部服務：gRPC

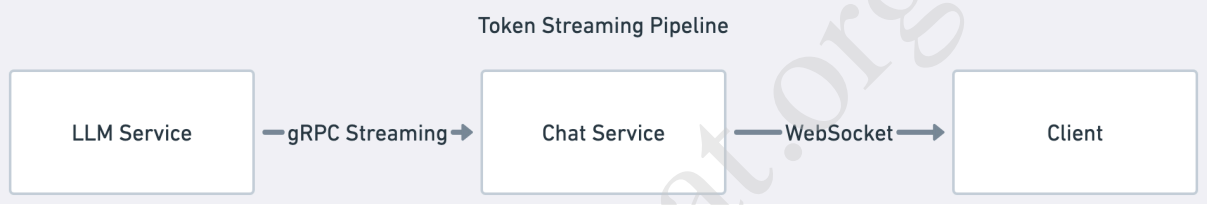
- Chat Service 的流程是嚴格循序的（讀歷史 → 改寫 query → 檢索 → 生成回答），每一步都依賴前一步的結果，屬於標準的同步 request-response 模式。

buildmoat.org

- 內部服務間採用 **gRPC**，相較於 REST 的優勢在於：基於 HTTP/2 的二進位協定延遲更低，且透過 protobuf 定義強型別的 service contract，跨團隊協作時減少溝通成本。

例外：Chat Service ↔ LLM Service 需要 streaming

- LLM 逐 token 生成回答。若 Chat Service 等待 LLM 生成完整回答後才回傳給 Client，使用者會經歷 2-3 秒的空白等待。
- 更好的做法是建立 **streaming pipeline**：LLM Service 以 gRPC server streaming 逐 token 推送給 Chat Service，Chat Service 再即時透過 WebSocket 轉發給 Client。
- 使用者看到的效果是文字逐字出現，time-to-first-token 從 2-3 秒降到數百毫秒，大幅降低感知延遲。



buildmoat.org